ReSequel: Robust LLM-assisted Query Rewriting and Optimization using Templatization and Sampling

Paper-ID: 996

Abstract

Heuristic query rewriting has long complemented cost-based optimization to improve query performance and robustness. Such rewrites transform SQL queries into semantically equivalent forms that are easier or faster to execute. Examples include standardizing and simplifying expressions, eliminating redundancy and common subexpressions, propagating constants, pushing down selections and projections, unnesting queries, and utilizing constraints. Modern DBMSs implement thousands of such rules, but maintaining and extending them is notoriously difficult. The interactions among rules are complex, and their static nature and application order prevent adaptation to specific database characteristics. Recent approaches that use large language models (LLMs) for learned query rewriting show promise but face challenges. They struggle with the large search space of possible rewrites, lack reliable query-equivalence verification, and fail to exploit metadata and statistics. We present **ReSequel**, an *outer optimization layer* on top of existing DBMSs to rewrite SQL queries through metadata-guided interaction with LLMs. ReSequel leverages catalog and statistical metadata to infer *template-specific rules* that guide the LLM toward effective transformations beyond the DBMS's fixed rule set. It automatically generates, verifies, and ranks rewritten query variants on sampled data in order to ensure semantic equivalence and measurable runtime gains. Our experiments cover five benchmark workloads: TPC-H, Stats, Public BI, IMDB, and StackOverflow. We evaluate ReSequel across multiple DBMSs, including PostgreSQL, MySQL, and DuckDB, and compared it with LLM-based query rewriting baselines. ReSequel achieves up to 3–14× speedups over native DBMSs and 5× over LLM-based systems, demonstrating scalable and stable query optimization across diverse workloads.

ACM Reference Format:

Paper-ID: 996. 2018. ReSequel: Robust LLM-assisted Query Rewriting and Optimization using Templatization and Sampling. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation emai (Conference acronym 'XX)*. ACM, New York, NY, USA, 14 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

After the introduction of the relational model by Ted Codd [14], QUEL (in Ingres [27], based on tuple calculus) and SEQUEL [10, 11] (in System R [4], based on relational algebra) were competing, early relational query languages. Ultimately, SEQUEL first became the de-facto standard and later evolved to the ANSI/ISO SQL (Structured Query Language) standard. The success of SQL and the relational model—despite criticism on specific aspects [8, 45]—was largely due to its (1) declarativity (what not how), (2) flexibility (compose arbitrary complex queries), (3) freedom for automatic optimization, and (4) physical data independence. Despite these compelling properties, all DBMS remain stubbornly sensitive to suboptimal query formulations because query optimization is "a never-ending quest for an increasingly better model and repertoire of optimization and execution techniques" (Patricia G. Selinger) [57], various shortcomings and simplifying assumptions (e.g., the independence assumption and its implication for redundant, correlated predicates) [40], and brittle rewrites (e.g., for unnesting sub-queries) [44].

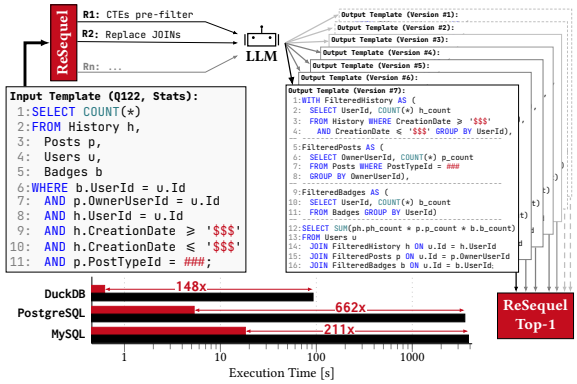


Figure 1: Comparison of ReSequel's Rewriting Impact on Host DBMS Performance. ReSequel rewrites masked templates into multiple variants using LLMs, and then locally verifies them, and selects the Top-1 Reconstructed Query.

Rule-based Query Rewriting: Many commercial or open-source DBMS have dedicated rule-based query rewriting systems, which transform queries into equivalent, but more efficient forms. These traditional rule engines date back to the early 1990s [48, 49], contain 1,000s of rules, and are applied heuristically before the cost-based optimization of join orders and physical operators. Example rules are the elimination of `DISTINCT` duplicate elimination if the result attributes contain an attribute with unique constraint, query unnesting, and the propagation of constants over joins:

$$R \bowtie_{R.a=S.b} \sigma_{S.b>7}(S) \rightarrow \sigma_{R.a>7}(R) \bowtie_{R.a=S.b} \sigma_{S.b>7}(S) \quad (1)$$

which enables prefiltering both join sides. Such rules are manually crafted by database experts over decades [7] or automatically discovered by specialized tools [5, 56]. Since the rules are expressed as source-target equivalences of small patterns, whole-query rewriting requires the iterative application of rules sets until a fixpoint or a maximum number of iterations. The interaction of these many heuristic rules together with heuristic phase ordering (sequence of applied rules) renders development and debugging difficult and

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/18/06
https://doi.org/XXXXXXX.XXXXXXX